# Оптимизация образов, безопасность и подготовка к Production

USER, ограничения ресурсов, multi-stage builds, кэш и анализ уязвимостей

# Цель и результаты занятия

- Объяснять угрозы при запуске приложения с лишними привилегиями.
- Собирать минимальный production-образ без инструментов, требующихся только при сборке.
- Проверять фактические размеры, права пользователя и работоспособность образов.
- Отличать обнаруженную CVE от доказанной эксплуатируемой уязвимости.

# Учебные вопросы

| № | Учебный вопрос |
|---|---|
| 1 | Безопасность контейнеров и минимизация привилегий |
| 2 | Оптимизация Dockerfile и multi-stage builds |
| 3 | Сканирование образов, доверенные источники и версии |

# Переход от development к production

- Development: удобная отладка, частые изменения кода, дополнительные утилиты и зависимости.
- Production: минимальные необходимые зависимости, явная конфигурация и контролируемый запуск.
- Разрабатываемый ранее стек сохраняется: Nginx → Express API → PostgreSQL.
- Изменяем преимущественно контейнер API; том PostgreSQL не удаляем.

# Какие риски уменьшаем

### Сборка без оптимизации

- Больше пакетов и инструментов в runtime.
- Приложение потенциально работает от root.
- Лишние файлы в контексте сборки.

### Целевая production-конфигурация

- Только runtime-зависимости.
- Непривилегированный пользователь.
- Ограничения ресурсов и прав.
- Проверяемая версия образа.

# Безопасность контейнеров и минимизация привилегий

Права, файловая система, ресурсные ограничения

# Схема: Поверхность атаки контейнера и принцип наименьших привилегий

Как уменьшить риски при запуске контейнеров Docker

## Небезопасно

**Большая поверхность атаки**




- USER root
- privileged
- запись во всю ФС
- лишние порты
- секреты в образе
- без лимитов CPU/RAM

## Принцип наименьших привилегий

1. **Не запускать от root**
   Использовать непривилегированного пользователя (node, app и т.п.).
2. **Давать только необходимые права**
   Запрещать лишние привилегии и возможности ядра.
3. **Ограничивать ресурсы**
   Устанавливать лимиты CPU и RAM, чтобы избежать DoS.
4. **Минимизировать файловую систему и порты**
   Только нужные томаr, read-only ФС, минимальный набор открытых портов.
5. **Хранить конфигурацию отдельно от образа**
   Использовать переменные окружения и внешние хранилища секретов.

## Правильно

**Минимальная поверхность атаки**




- USER node / app
- только нужные порты
- read_only
- tmpfs /tmp
- no-new-privileges
- лимиты CPU/RAM
- переменные окружения вместо секретов в образе
- только нужные тома
- минимальные права

## Что важно понять

- **Контейнер — не граница абсолютной безопасности**
  Контейнер изолирует, но не заменяет другие меры защиты.
- **Root в контейнере повышает риск**
  Компрометация приложения может привести к большей зоне влияния.
- **Меньше прав — меньше последствий ошибки**
  Ограниченные привилегии снижают ущерб при эксплуатации уязвимостей.
- **Безопасность и производительность нужно проверять на практике**
  Тестируйте, профилируйте и подбирайте настройки под реальные нагрузки и сценарии.

## Примеры настроек (Dockerfile / docker-compose)

```
USER node
```

```
read_only: true
```

```
tmpfs:
  - /tmp
```

```
cap_drop:
  - ALL
```

```
security_opt:
  - no-new-privileges:true
```

# Контейнер не равен полноценной виртуальной машине

- Процессы Linux-контейнера используют общее ядро хоста.
- Контейнерная изоляция снижает воздействие, но не отменяет опасных настроек и уязвимостей ядра.
- Не запускайте без необходимости --privileged, не монтируйте Docker socket и каталоги хоста.
- Управление Docker Engine на хосте требует повышенных привилегий.

# Root внутри контейнера: почему это риск

- В некоторых образах пользователь по умолчанию имеет UID 0.
- Изоляция и capabilities ограничивают процесс, но root расширяет последствия ошибок конфигурации.
- Проводите runtime-процессы от отдельного непривилегированного пользователя.
- Требуемые файлы и каталоги должны быть доступны для чтения/записи нужному UID.

# Пользователь в Dockerfile

DOCKERFILE

```
FROM node:22-bookworm-slim
WORKDIR /app
COPY --chown=node:node . .
USER node
CMD ["node", "app.js"]
```

**Что важно**

- USER node задаёт пользователя для последующих инструкций и процесса запуска.
- COPY --chown обеспечивает нужное владение файлами.
- Такой пример демонстрирует права, но ещё не является оптимальной схемой установки зависимостей.

# Проверка пользователя контейнера

BASH

```
docker run --rm express-api:prod id
docker inspect --format "{{.Config.User}}" express-api:prod
```

**Что важно**

- id показывает фактический UID и GID.
- Config.User показывает значение в конфигурации образа; пустое значение не доказывает отсутствие root.

# Принцип наименьших привилегий

- Не публикуйте порт PostgreSQL без необходимости: доступ между сервисами работает во внутренней сети Compose.
- Не храните секреты в исходном коде, Dockerfile и слоях образа.
- Не включайте лишние Linux capabilities и привилегированный режим.
- Механизмы read_only, cap_drop и no-new-privileges сокращают возможности процесса, но требуют тестирования.

# Ограничение CPU и памяти

BASH

```
docker run --rm --cpus=0.5 --memory=256m express-api:prod
docker stats --no-stream
```

**Что важно**

- CPU limit 0.5 — ограничение эквивалентом половины одного CPU.
- 256m — жёсткий предел памяти контейнера; при превышении возможен OOM kill.
- Фактическое потребление проверяют через stats.

# Лимиты и права в Compose

```yaml
services:
  api:
    mem_limit: 256m
    cpus: 0.50
    read_only: true
    tmpfs:
      - /tmp
    security_opt:
      - no-new-privileges:true
```

**Что важно**

- Ограничения применяются к api, а не ко всем сервисам.
- read_only закрывает запись в корневой слой; tmpfs оставляет временный каталог.
- При требовании записи в другие места нужны отдельные mounts.

# Схема: Непривилегированный контейнер и ограничения ресурсов

Как запуск от непривилегированного пользователя и лимиты CPU/RAM повышают безопасность и стабильность Docker-приложения

## Неправильно

Рискованный вариант




**USER root**
запуск от привилегированного пользователя

**без лимитов CPU/RAM**
может использовать все ресурсы хоста

**риск эскалации привилегий**
при уязвимости в приложении или зависимостях

**DoS / исчерпание ресурсов**
может перегрузить хост и соседние сервисы

**ошибка приложения влияет сильнее**
может привести к сбоям на хосте

**выше поверхность атаки**
больше возможностей для злоумышленника

Перегрузка CPU




Исчерпание памяти




Может повлиять на хост и другие контейнеры

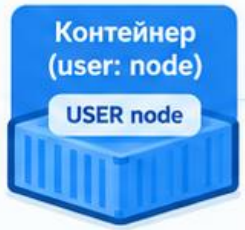
## Два ключевых способа защиты

### 1 Непривилегированный пользователь

Запуск контейнера от обычного пользователя

**Dockerfile**

```
#...
USER node
# или
USER app
```




- процесс не root
- меньше привилегий
- ниже риск компрометации

### 2 Ограничение ресурсов

Лимиты на использование CPU и оперативной памяти

**docker-compose.yml**

```
services:
  app:
    mem_limit: 256m
    cpus: 0.50
```

или

**Команда docker run**

```
docker run \
  ---memory=256m \
  --cpus=0.5 \
  myapp:latest
```




**Лимит CPU**
(например, 0.5)

0.5 CPU
(50% одного ядра)




**Лимит RAM**
(например, 256m)

256 MB
(ограничение памяти)

## Правильно

Безопасный и стабильный вариант




**USER node**
запуск от непривилегированного пользователя

**cpus: 0.5**
ограничение CPU

**memory: 256m**
ограничение памяти

**read_only (опционально)**
файловая система только для чтения

**no-new-privileges**
запрет на повышение привилегий

**меньше риск**
при компрометации приложения

**предсказуемая нагрузка**
контроль использования ресурсов

**стабильнее на сервере**
не влияет на другие сервисы

## Что важно понять

1. Запуск не от **root** уменьшает потенциальный ущерб при взломе приложения.
2. **Лимиты CPU и RAM** защищают хост и соседние сервисы от перегрузки.
3. Ограничения ресурсов не заменяют исправление ошибок в коде, но снижают последствия.
4. Безопасность и производительность нужно проверять тестами и мониторингом.

## Примеры конфигурации

**Dockerfile**

```
# Базовый образ
FROM node:20

# Создание пользователя (если нужно)
RUN useradd -m app

# Запуск от непривилегированного пользователя
USER node
```

**docker-compose.yml**

```
services:
  app:
    image: myapp:latest
    mem_limit: 256m
    cpus: 0.50
    #   опционально:
    read_only: true
    security_opt:
      - no-new-privileges:true
```

**docker run** (опционально)

```
docker run \
    --memory=256m \
    --cpus=0.5 \
    --read-only \
    --security-opt
    no-new-privileges:true \
    myapp:latest
```

# Read-only filesystem и временные данные

- Флаг --read-only запрещает изменения в корневой файловой системе контейнера.
- Это не делает автоматически read-only именованные тома и отдельные mounts.
- Если приложению нужен /tmp, задайте --tmpfs /tmp или tmpfs в Compose.
- PostgreSQL со своим томом необходимо настраивать отдельно, не копируя настройки API механически.

# Проверка режима только для чтения

BASH

```
docker run --rm --read-only --tmpfs /tmp express-api:prod
docker run --rm --read-only --entrypoint sh express-api:prod -c "touch /app/test"
```

**Что важно**

- Первая команда запускает API; её останавливают Ctrl+C в интерактивном режиме.
- Вторая предсказуемо показывает невозможность создавать файл в /app при read-only rootfs.

# Чего не делать с Docker permissions

- Не включайте --privileged для решения ошибок доступа к файлам.
- Не используйте chmod 777 на каталоги приложения как универсальное исправление.
- Участие в группе docker на хосте предоставляет практически root-уровень контроля над Docker Engine.
- Для учебной среды можно продолжить использовать sudo docker, как в предыдущих лабораторных.

# Оптимизация Dockerfile и multi-stage builds

Состав runtime-образа, кэширование, контекст сборки

# Схема: Multi-stage build — builder и runtime

Как многоэтапная сборка помогает отделить этап **сборки приложения** от финального production-образа

## 1 Этап builder

Полноценное окружение для сборки приложения

**node:22 (builder)** — БОЛЬШЕЙ РАЗМЕР (временный образ)

- **Исходный код приложения** — `src/`
- **package.json** — Зависимости проекта
- **npm зависимости** — devDependencies (все зависимости)
- **Инструменты сборки** — компиляторы, bundler и т.д.

```
npm ci
npm run build
```

→ **Собранное приложение** (dist/ и другие артефакты)

В этом этапе есть всё необходимое для сборки:
исходный код, devDependencies, компиляторы, инструменты сборки.
Образ может быть большим и используется только на этапе сборки.

**COPY --from=builder**

Копируем только нужные файлы (артефакты сборки)

→ **dist/** и другие production-файлы

## 2 Этап runtime

Минимальный и чистый образ для запуска

**node:22-bookworm-slim (runtime)** — МЕНЬШИЙ РАЗМЕР (готовый production-образ)

- **Файлы приложения (собранные)** — `dist/`
- **Только production-зависимости** — `node_modules` (production)
- **Node.js (рантайм)**
- **Пользователь приложения** — node (не root)

**Не включаем (исключено из финального образа):**

- ✕ исходный код проекта
- ✕ инструменты сборки и компиляторы
- ✕ devDependencies и лишние файлы

Финальный образ содержит только необходимое для запуска приложения.
Он меньше по размеру, безопаснее и быстрее разворачивается.

## Пример Dockerfile

```dockerfile
# Этап builder
FROM node:22 AS builder
WORKDIR /app

COPY package*.json ./
RUN npm ci

COPY . .
RUN npm run build

# Этап runtime
FROM node:22-bookworm-slim
WORKDIR /app

COPY --from=builder /app /app

USER node
CMD ["node", "app.js"]
```

## Без multi-stage

Один большой образ, содержащий всё

**node:22 (один образ)**

Исходный код + Инструменты сборки + devDependencies (все зависимости) + Собранное приложение

- Большой размер образа (сотни МБ)
- Больше поверхность атаки
- Дольше передача и развертывание

## С multi-stage

Разделение на builder и runtime

**builder** (временный)

Исходный код + инструменты + devDependencies

→ **runtime** (финальный)

Только нужные файлы + production-зависимости + рантайм

- Меньший размер образа (десятки МБ)
- Меньше лишних файлов и зависимостей
- Меньше поверхность атаки
- Быстрее передача и развертывание

## Что важно понять

1. Builder нужен для сборки, а runtime — для запуска.
2. В финальный образ копируют только нужные файлы.
3. Multi-stage уменьшает размер образа и поверхность атаки.
4. Для production лучше использовать отдельный runtime-этап.

## Преимущества

- меньше размер образа
- меньше лишних зависимостей
- удобнее для production

## Типичные ошибки

- копировать весь проект в runtime
- оставлять devDependencies
- хранить секреты в образе

# Откуда появляется большой размер

- Базовый образ несёт собственную ОС-часть и системные библиотеки.
- node_modules может содержать development-зависимости и ненужные файлы.
- Установленные инструменты компиляции часто не нужны при запуске.
- Контекст сборки может случайно включить .git, .env и локальные node_modules.

# Базовые образы: компромиссы

**node:22 / bookworm-slim**

- Обычный Debian-вариант и уменьшенная Debian-сборка.
- Больше совместимости с glibc-зависимостями.
- Измеряйте размер и CVE на конкретной дате.

**node:22-alpine / distroless**

- Alpine использует musl вместо glibc.
- Для некоторых нативных модулей могут потребоваться дополнительные действия.
- Distroless сокращает инструменты в runtime, усложняя привычную интерактивную диагностику.

# Пример базового Dockerfile

DOCKERFILE

```
FROM node:22
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "app.js"]
```

**Что важно**

- Один этап, полный Node-образ и npm install — учебный baseline.
- npm install учитывает devDependencies; используем для сравнения, а не как production-эталон.

# Multi-stage builds: главная идея

- Каждый FROM начинает новый этап сборки.
- Этап builder/deps получает инструменты и зависимости для сборки.
- Runtime-этап копирует только нужные артефакты и production-зависимости.
- Для чистого Express без компиляции выигрыш от multi-stage может быть небольшим: размер определяет прежде всего runtime-base и зависимости.

# Production Dockerfile: два этапа

DOCKERFILE

```
FROM node:22-bookworm-slim AS deps
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev

FROM node:22-bookworm-slim AS runtime
ENV NODE_ENV=production
WORKDIR /app
COPY --from=deps --chown=node:node /app/node_modules ./node_modules
COPY --chown=node:node . .
USER node
EXPOSE 3000
CMD ["node", "app.js"]
```

**Что важно**

- Для npm ci обязательно наличие согласованного package-lock.json.
- Первый этап формирует только production-зависимости.
- Во втором этапе не переносится npm cache; добавление полного исходного каталога контролирует .dockerignore.

# Файл .dockerignore

```text
node_modules
.git
.env
.env.*
!.env.example
*.log
coverage
Dockerfile.baseline
Dockerfile.production
```

**Что важно**

- Исключённые файлы не попадают в build context.
- Секреты следует исключать и не создавать их внутри слоёв образа.
- Обратите внимание: точечные исключения нужны, если файл действительно требуется сборке.

# Схема: Кэширование npm-зависимостей и .dockerignore

## 1. Правильный порядок инструкций

**Dockerfile**

```
1  FROM node:22-bookworm-slim
2  WORKDIR /app
3  COPY package*.json ./
4  RUN npm ci --omit=dev
5  COPY . .
6  RUN npm run build
```

Слой кэшируется, если зависимости не меняются

Пересобирается только при изменении кода приложения

## 2. Как работает кэширование слоёв

**COPY package*.json**
Создаётся слой с файлами зависимостей
→ Слой 1 (кэшируется)

↓

**RUN npm ci**
Устанавливаются npm-зависимости
→ Слой 2 (кэшируется)

↓

**COPY . .**
Копируется остальной код приложения
→ Слой 3 (пересобирается при изменении кода)

↓

**RUN npm run build**
Сборка приложения
→ Слой 4 (пересобирается при изменении кода)

## 3. .dockerignore — что исключаем из контекста

**.dockerignore**

```
# Зависимости и артефакты сборки
node_modules
dist

# Конфиденциальные файлы
.env
*.pem

# Системные и служебные файлы
.git
.DS_Store
docker-compose*.yml
npm-debug.log
tests
docs
```

**Преимущества**

- Меньший размер контекста сборки
- Более быстрая сборка
- Меньше слоёв и меньше образ
- Исключение конфиденциальных данных
- Меньшая вероятность утечки секретов в образ

## 4. Что попадает в контекст сборки

**Без .dockerignore**

```
project/
  node_modules/   250 МБ
  dist/            80 МБ
  .git/           120 МБ
  src/              5 МБ
  ...
```

Контекст: ~ 455 МБ

Применяем .dockerignore

**С .dockerignore**

```
project/
  src/              5 МБ
  package.json      1 КБ
  package-lock.json 500 КБ
  Dockerfile        1 КБ
  ...
```

Контекст: ~ 5–10 МБ

## 5. Рекомендации

**Разделяйте COPY**
Сначала package*.json, затем остальной код

**Используйте npm ci**
Быстрая и воспроизводимая установка зависимостей

**.dockerignore**
Исключайте ненужные и конфиденциальные файлы

**Проверяйте кэш**
Используйте --progress=plain для анализа слоёв сборки

**Контролируйте размер**
Проверяйте размер контекста и итогового образа ( docker build ., docker images )

# Кэш: порядок COPY и RUN

- Сперва копируйте package.json и package-lock.json.
- Следующим шагом выполняйте npm ci; слой зависит от манифестов.
- Лишь затем копируйте исходный код приложения.
- Изменение app.js не обязано повторно запускать npm ci при неизменённых манифестах.

# Сборка и сравнение двух образов

BASH

```
docker build -f Dockerfile.baseline -t express-api:baseline ./api
docker build -f Dockerfile.production -t express-api:prod ./api
docker image ls express-api
docker history express-api:prod
```

**Что важно**

- Команды запускаются из корня проекта; Dockerfile расположены в каталоге api.
- Тег prod — учебное имя, не доказательство готовности к production.
- Размеры и слои фиксируем после реальной сборки.

# Два независимых свойства: размер и безопасность

- Небольшой образ не обязательно имеет меньше критических проблем.
- Меньшее количество пакетов сокращает поверхность потенциальных уязвимостей, но не подтверждает отсутствие CVE.
- Размеры из демонстраций не универсальны: они зависят от базовых тегов, зависимостей и архитектуры.
- Измерение до/после — обязательная часть учебного эксперимента.

# Сканирование уязвимостей и управление версиями

Результаты аудита необходимо интерпретировать

# Схема: Сканирование Docker-образов и анализ CVE

## 1 Выбираем образ для проверки

**Dockerfile**

```
FROM node:22
COPY ...
RUN npm ci ...
```

**Docker-образ**
(локальный или из реестра)

```
shop-api:latest
node:22
node:22-alpine
<registry>/shop-api:1.0.0
```

## 2 Запускаем сканирование

**Docker Scout**
Анализ уязвимостей (CVE)
и рекомендации по исправлению

```
# Быстрый обзор образа
docker scout quickview shop-api:latest
# Подробный анализ
docker scout cves shop-api:latest
# Можно проверить образ из реестра
docker scout quickview node:22-alpine
```

## 3 Получаем и анализируем результаты

```
docker scout quickview shop-api:latest
```

| Серьёзность | Количество | Процент |
|---|---|---|
| Критические (Critical) | 2 | 4% |
| Высокие (High) | 8 | 16% |
| Средние (Medium) | 28 | 56% |
| Низкие (Low) | 12 | 24% |

**Всего уязвимостей: 50**

## 4 Детальная информация по CVE

```
docker scout cves shop-api:latest
```

| ID | SEVERITY | PACKAGE | INSTALLED | FIXED IN |
|---|---|---|---|---|
| CVE-2024-3094 | Critical | openssl | 3.0.11 | 3.0.13 |
| CVE-2023-46809 | High | nodejs | 22.1.0 | 22.3.1 |
| CVE-2024-6387 | High | glibc | 2.36 | 2.37 |
| CVE-2024-1208 | Medium | libpng | 1.6.37 | 1.6.40 |
| CVE-2023-4863 | Medium | libwebp | 1.3.2 | 1.3.3 |

- **ID** — Идентификатор уязвимости (CVE)
- **SEVERITY** — Уровень опасности
- **PACKAGE** — Уязвимый пакет (библиотека)
- **INSTALLED** — Установленная версия
- **FIXED IN** — Версия с исправлением

## 5 Что делать дальше

**Обновить базовый образ**
Использовать актуальные версии
(node:22 → node:22-bookworm-slim)

**Обновить зависимости**
npm audit, npm update,
использовать npm ci

**Пересобрать образ**
docker build --no-cache -t shop-api:1.0.1 .

**Проверить снова**
Повторное сканирование для
подтверждения устранения уязвимостей

## 6 Сравнение образов (пример)

**node:22 (базовый образ)**

| | |
|---|---|
| Критические | 5 |
| Высокие | 18 |
| Средние | 42 |
| Низкие | 20 |

**Всего: 85**

**Большой размер и больше уязвимостей**

**node:22-alpine**

| | |
|---|---|
| Критические | 0 |
| Высокие | 4 |
| Средние | 12 |
| Низкие | 8 |

**Всего: 24**

**Меньший размер и меньше уязвимостей**

## 7 Рекомендации

**Используйте официальные образы**
от проверенных поставщиков
(Docker Hub, официальные репозитории)

**Регулярно обновляйте**
базовые образы и зависимости

**Сканируйте образы**
в составе CI/CD
(например, Docker Scout, Trivy)

**Исправляйте уязвимости**
с высоким и критическим
уровнем в первую очередь

**Используйте теги и digest**
избегайте только тега latest,
фиксируйте версию образа

# Что такое CVE, CVSS и исправление

- CVE — идентификатор опубликованной уязвимости.
- CVSS помогает оценивать тяжесть, но сам по себе не определяет реальный риск эксплуатации.
- Результат зависит от базы уязвимостей и даты сканирования.
- Учитывайте применимость к библиотеке, наличие патча и внешнюю доступность компонента.

# Проверка Docker Scout

```bash
docker scout version
docker scout quickview express-api:baseline
docker scout quickview express-api:prod
```

**Что важно**

- Scout CLI может отсутствовать на Linux-сервере: предварительно проверьте доступность команды.
- Результаты quickview записывают с датой, уровнем серьёзности и названием образа.
- Для расширенного списка используют docker scout cves.

# Если Docker Scout не установлен

- Преподаватель может заранее подготовить демонстрационную машину с установленным Scout CLI.
- Либо применить другой доступный сканер, например Trivy, если его установка предусмотрена в учебном классе.
- Не требуется устанавливать сканер в работающий production-контейнер.
- Отсутствие сканера не препятствует выполнению базовой части про USER, размеры, лимиты и read-only.

# Официальные и доверенные источники

- Проверяйте издателя, документацию и политику обновления образа.
- Избегайте непроверенных сторонних образов с неизвестным содержимым.
- Обновляйте базовую ОС и npm-зависимости по контролируемому процессу.
- Не включайте секреты и токены при публикации Docker-образов.

# Теги и digest

```bash
docker image inspect express-api:prod --format "{{.Id}}"
docker image ls --digests
docker image tag express-api:prod express-api:1.0.0
```

**Что важно**

- SemVer-тег полезен человеку, но может быть переназначен.
- Image ID — локальный идентификатор содержимого, digest (если есть) адресует конкретный манифест.
- У локально созданного и не опубликованного образа RepoDigests может отсутствовать.

# Рекомендованная проверка перед выпуском

| Проверка | Пример действия | Что фиксировать |
| --- | --- | --- |
| Пользователь | docker run ... id | UID != 0 |
| Привилегии | read_only, security_opt | Ограничения соответствуют поведению приложения |
| Размер | docker image ls | Фактический размер, а не обещание |
| Уязвимости | docker scout quickview | Дата, тяжесть CVE, возможные исправления |
| Работоспособность | GET /api/products | HTTP-статус и JSON-ответ |

# Практическая демонстрация на лекции

| | | |
|---|---|---|
| 1 | **Шаг 1** | Собрать baseline и production образы из одних исходников. |
| 2 | **Шаг 2** | Сравнить размеры, историю слоёв и пользователи запуска. |
| 3 | **Шаг 3** | Запустить оба по очереди с одинаковыми переменными окружения и проверить HTTP. |
| 4 | **Шаг 4** | Ограничить CPU/RAM и включить read_only для API в Compose. |
| 5 | **Шаг 5** | При доступности Scout сравнить отчёты; объяснить, почему число CVE может не уменьшиться. |

# Типичные ошибки при оптимизации

- npm ci не работает без package-lock.json или при несогласованных манифестах.
- USER node не может записать файл в директорию root:root — проверьте права.
- read_only ломает приложение, записывающее временные файлы вне tmpfs.
- Alpine может вызвать проблемы с нативными библиотеками: выбирайте базу по совместимости.
- Пересоздание API не требует удаления тома с данными PostgreSQL.

# Контрольные вопросы

1. Чем опасен root внутри контейнера?
2. Почему group docker на хосте обладает высокими привилегиями?
3. Для чего нужны ограничения CPU и RAM?
4. Что меняет read_only и какую роль выполняет tmpfs?
5. В каких случаях multi-stage особенно полезен?
6. Чем npm ci --omit=dev отличается от npm install?
7. Зачем копировать package*.json перед исходным кодом?
8. Почему .dockerignore важен для защиты секретов?
9. Почему меньший размер образа не гарантирует меньше CVE?
10. Чем отличаются tag, Image ID и digest?

# Итоги лекции

1. Production-образ должен запускать приложение с минимально необходимыми правами.
2. Ограничения ресурсов и read_only проверяют на работающем проекте.
3. Multi-stage отделяет этап подготовки зависимостей от runtime; результат измеряют.
4. .dockerignore уменьшает контекст и предотвращает случайное копирование файлов.
5. Сканирование CVE дополняет, но не заменяет обновления и тестирование.
6. Изменяя API, не удаляем том PostgreSQL и не нарушаем Nginx proxy.